Implementing Login Functionality in ASP.NET Core

Prerequisite

- **Stored Procedure:** `PR_User_Login`

Stored Procedure Code:

```
CREATE PROCEDURE [dbo].[PR_User_Login]
    @UserName NVARCHAR(50),
    @Password NVARCHAR(50)
AS
BEGIN
    SELECT
        [dbo].[User].[UserID],
        [dbo].[User].[UserName],
        [dbo].[User].[MobileNo],
        [dbo].[User].[Email],
        [dbo].[User].[Password],
        [dbo].[User].[Address]
    FROM
        [dbo].[User]
    WHERE
        [dbo].[User].[UserName] = @UserName
        AND [dbo].[User].[Password] = @Password;
END
```

Step 1: Layout Setup in `_Layout_Login.cshtml`

Ensure that your layout file includes the necessary CSS and JS libraries to support the login page.

Code:

```
<!DOCTYPE  html>
<html  lang="en">
<head>
    <meta  charset="utf-8">
    <meta  content="width=device-width, initial-scale=1.0"  name="viewport">
    <title>Pages / Login - NiceAdmin Bootstrap Template</title>
    <meta  content=""  name="description">
    <meta  content=""  name="keywords">

    <!-- Favicons -->
    <link  href="~/img/favicon.png"  rel="icon">
    <link  href="~/img/apple-touch-icon.png"  rel="apple-touch-icon">

    <!-- Google Fonts -->
```

```
    <link  href="https://fonts.gstatic.com"  rel="preconnect">
    <link  href="https://fonts.googleapis.com/css?
family=Open+Sans:300,300i,400,400i,600,600i,700,700i|Nunito:300,300i,400,400i,600,
600i,700,700i|Poppins:300,300i,400,400i,500,500i,600,600i,700,700i"
rel="stylesheet">
    <!-- Vendor CSS Files -->
    <link  href="~/vendor/bootstrap/css/bootstrap.min.css"  rel="stylesheet">
    <link  href="~/vendor/bootstrap-icons/bootstrap-icons.css"  rel="stylesheet">
    <link  href="~/vendor/boxicons/css/boxicons.min.css"  rel="stylesheet">
    <link  href="~/vendor/quill/quill.snow.css"  rel="stylesheet">
    <link  href="~/vendor/quill/quill.bubble.css"  rel="stylesheet">
    <link  href="~/vendor/remixicon/remixicon.css"  rel="stylesheet">
    <link  href="~/vendor/simple-datatables/style.css"  rel="stylesheet">
    <!-- Template Main CSS File -->
    <link  href="~/css/style.css"  rel="stylesheet">
    <!-- =======================================================
    * Template Name: NiceAdmin
    * Updated: May 30 2023 with Bootstrap v5.3.0
    * Template URL: https://bootstrapmade.com/nice-admin-bootstrap-admin-html-
template/
    * Author: BootstrapMade.com
    * License: https://bootstrapmade.com/license/
    ======================================================== -->
</head>
<body>
@RenderBody()
<a  href="#"  class="back-to-top d-flex align-items-center justify-content-
center"><i  class="bi bi-arrow-up-short"></i></a>

<!-- Vendor JS Files -->
<script  src="~/vendor/apexcharts/apexcharts.min.js"></script>
<script  src="~/vendor/bootstrap/js/bootstrap.bundle.min.js"></script>
<script  src="~/vendor/chart.js/chart.umd.js"></script>
<script  src="~/vendor/echarts/echarts.min.js"></script>
<script  src="~/vendor/quill/quill.min.js"></script>
<script  src="~/vendor/simple-datatables/simple-datatables.js"></script>
<script  src="~/vendor/tinymce/tinymce.min.js"></script>
<script  src="~/vendor/php-email-form/validate.js"></script>
<!-- Template Main JS File -->
<script src="~/js/main.js"></script>
<script src="~/lib/jquery/dist/jquery.min.js"></script>
<script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
@await RenderSectionAsync("Scripts", required: false)
</body>
</html>
```

Step 2: Create the `UserLoginModel` in the UserModel

Create a model that represents the login form data (Username and Password), and add data annotations for validation.

Code:

```
public class UserLoginModel
{
    [Required(ErrorMessage = "Username is required.")]
    public string UserName { get; set; }

    [Required(ErrorMessage = "Password is required.")]
    public string Password { get; set; }
}
```

Step 3: Implement the Login Logic in `UserController.cs`

Handle the login process in the `UserController`. It uses the stored procedure `PR_User_Login` to validate user credentials. If the credentials are correct, it stores the user data in the session.

Code:

```
public IActionResult UserLogin(UserLoginModel userLoginModel)
{
    try
    {
        if (ModelState.IsValid)
        {
            string connectionString =
this.configuration.GetConnectionString("ConnectionString");
            SqlConnection sqlConnection = new SqlConnection(connectionString);
            sqlConnection.Open();
            SqlCommand sqlCommand = sqlConnection.CreateCommand();
            sqlCommand.CommandType = System.Data.CommandType.StoredProcedure;
            sqlCommand.CommandText = "PR_User_Login";
            sqlCommand.Parameters.Add("@UserName", SqlDbType.VarChar).Value =
userLoginModel.UserName;
            sqlCommand.Parameters.Add("@Password", SqlDbType.VarChar).Value =
userLoginModel.Password;
            SqlDataReader sqlDataReader = sqlCommand.ExecuteReader();
            DataTable dataTable = new DataTable();
            dataTable.Load(sqlDataReader);
            if (dataTable.Rows.Count > 0)
            {
                foreach (DataRow dr in dataTable.Rows)
                {
                    HttpContext.Session.SetString("UserID",
dr["UserID"].ToString());
                    HttpContext.Session.SetString("UserName",
dr["UserName"].ToString());
                }

                return RedirectToAction("ProductList", "Product");
            }
            else
            {
```

```
                return RedirectToAction("Login", "User");
            }

        }
    }
    catch (Exception e)
    {
        TempData["ErrorMessage"] = e.Message;
    }

    return RedirectToAction("Login");
}
```

Step 3: Create the Login Page (`Login.cshtml`)

Set up your login page to accept the username and password. Use `asp-for` to bind the model properties and `asp-validation-for` to display validation errors.

Code:

```
@{
    Layout = "~/Views/Shared/_Layout_Login.cshtml";
}

@model CoffeeShop.Models.UserLoginModel

<main>
    <div class="container">
        @if (TempData["ErrorMessage"] != null)
        {
            <div class="alert alert-danger">
                @TempData["ErrorMessage"]
            </div>
        }
        <section class="section register min-vh-100 d-flex flex-column align-
items-center justify-content-center py-4">
            <div class="container">
                <div class="row justify-content-center">
                    <div class="col-lg-4 col-md-6 d-flex flex-column align-items-
center justify-content-center">

                        <div class="d-flex justify-content-center py-4">
                            <a class="logo d-flex align-items-center w-auto">
                                <img src="~/img/logo.png" alt="">
                                <span class="d-none d-lg-block">NiceAdmin</span>
                            </a>
                        </div><!-- End Logo -->

                        <div class="card mb-3">

                            <div class="card-body">
```

```
                                <div class="pt-4 pb-2">
                                    <h5 class="card-title text-center pb-0 fs-
4">Login to Your Account</h5>
                                    <p class="text-center small">Enter your
username & password to login</p>
                                </div>

                                <form class="row g-3 needs-validation" asp-
action="UserLogin" asp-controller="User">

                                    <div class="col-12">
                                        <label for="yourUsername" class="form-
label">Username</label>
                                        <div class="input-group has-validation">
                                            <span class="input-group-text"
id="inputGroupPrepend"></span>
                                            <input type="text" asp-for="UserName"
class="form-control" id="yourUsername">
                                            <span asp-validation-for="UserName"
class="text-danger"></span>
                                        </div>
                                    </div>

                                    <div class="col-12">
                                        <label for="yourPassword" class="form-
label">Password</label>
                                        <input type="password" asp-for="Password"
class="form-control" id="yourPassword">
                                        <span asp-validation-for="Password"
class="text-danger"></span>
                                    </div>

                                    <div class="col-12">
                                        <div class="form-check">
                                            <input class="form-check-input"
type="checkbox" name="remember" value="true" id="rememberMe">
                                            <label class="form-check-label"
for="rememberMe">Remember me</label>

                                        </div>
                                    </div>
                                    <div class="col-12">
                                        <button class="btn btn-primary w-100"
type="submit">Login</button>
                                    </div>
                                    <div class="col-12">
                                        <p class="small mb-0">Don't have account?
<a>Create an account</a></p>
                                    </div>
                                </form>

                            </div>
                        </div>
```

```
                    </div>
                </div>
            </div>

        </section>

    </div>
</main><!-- End #main -->

@section Scripts{
    @{
        await Html.RenderPartialAsync("_ValidationScriptsPartial");
    }
}
```

Step 5: Implement Logout in `UserController.cs`

Create an action to clear the session when the user logs out.

Code:

```
public IActionResult Logout()
{
    HttpContext.Session.Clear();
    return RedirectToAction("Login", "User");
}
```

Step 6: Add Session and Authorization Logic with `CheckAccess.cs`

To restrict access based on the session, implement `CheckAccess` to ensure that a user cannot access pages without logging in. (Create this file on prject level)

Code:

```
public class CheckAccess : ActionFilterAttribute, IAuthorizationFilter
{
    public void OnAuthorization(AuthorizationFilterContext filterContext)
    {
        if (filterContext.HttpContext.Session.GetString("UserID") == null)
        {
            filterContext.Result = new RedirectResult("~/User/Login");
        }
    }

    public override void OnResultExecuting(ResultExecutingContext context)
    {
        context.HttpContext.Response.Headers["Cache-Control"] = "no-cache, no-
```

```
store, must-revalidate";
        context.HttpContext.Response.Headers["Expires"] = "-1";
        context.HttpContext.Response.Headers["Pragma"] = "no-cache";
        base.OnResultExecuting(context);
    }
}
```

```
[CheckAccess]
public class HomeController : Controller
{
  // Rest of code for controller
}
```

If you place `[CheckAccess]` on a controller like HomeController, every action in that controller will run through the CheckAccess authorization logic. If you place `[CheckAccess]` on individual action methods, only those specific methods will use the custom authorization logic.

Step 7: Add Method For Accessing UserName,Address From Session in `CommonVariable.cs`

(Create this file on prject level)

Code:

```
public class CommonVariable
{
    private static IHttpContextAccessor _HttpContextAccessor;

    static CommonVariable()
    {
        _HttpContextAccessor = new HttpContextAccessor();
    }


    public static int? UserID()
    {

        if (_HttpContextAccessor.HttpContext.Session.GetString("UserID") == null)
        {
            return null;
        }

        return
Convert.ToInt32(_HttpContextAccessor.HttpContext.Session.GetString("UserID"));
    }

    public static string UserName()
```

```
    {
        if (_HttpContextAccessor.HttpContext.Session.GetString("UserName") ==
null)
        {
            return null;
        }

        return _HttpContextAccessor.HttpContext.Session.GetString("UserName");
    }

    public static string Email()
    {
        if (_HttpContextAccessor.HttpContext.Session.GetString("EmailAddress") ==
null)
        {
            return null;
        }
        return _HttpContextAccessor.HttpContext.Session.GetString("EmailAddress");
    }
}
```

Step 8: **Configure Session in Program.cs**

1. **Add Session Services:**

Add session configuration inside the builder.Services section.

Code:

```
var builder = WebApplication.CreateBuilder(args);

//Login
builder.Services.AddDistributedMemoryCache();
builder.Services.AddHttpContextAccessor();
builder.Services.AddSession();

builder.Services.AddControllersWithViews();

var app = builder.Build();
```

Enable Session in the Pipeline:

After configuring the services, make sure the session middleware is enabled in the request pipeline.

```
// Enable session middleware
app.UseSession(); // Must be added before app.MapControllerRoute

app.UseHttpsRedirection();
app.UseStaticFiles();
```

```
app.UseRouting();

// enables the authentication middleware in ASP.NET Core to handle user
authentication for securing endpoints.
app.UseAuthentication();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```